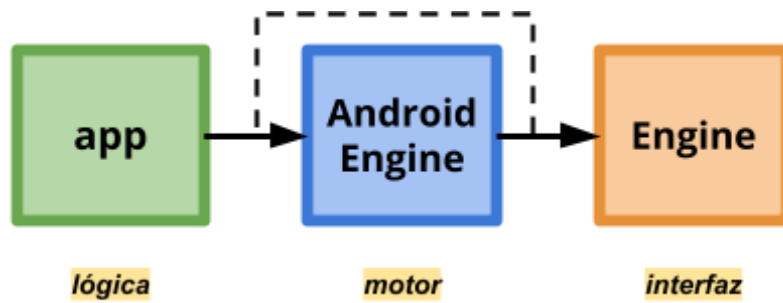


Práctica 2: Arquitectura

Adrián Migueles D'Ambrosio

Mario Miguel Cuartero

Simeón Konstantinov Petrov



ANDROID ENGINE sirve como un conjunto de clases que proporciona las implementaciones necesarias en Android e implementa las interfaces de Engine.

- **AndroidEngine:** Clase que gestiona el loop del programa y maneja la escena (renderizado, input, audio, archivos, anuncios, notificaciones), además carga una librería nativa c++ que permite encriptar archivos.
- **AndroidInput:** Clase que controla los eventos recogidos y adapta sus tipos en Android.
- **AndroidGraphics:** Clase que genera las gráficas en Android y permite el dibujo de todos sus elementos (*SurfaceView, Canvas, Paint, Bitmap, Typeface*).
- **AndroidImage:** Clase que permite generar imágenes en Android (Llamada desde AndroidGraphics).
- **AndroidFont:** Clase que crea fuentes en Android (*llamada también desde AndroidGraphics*).
- **AndroidAudio:** Clase que controla el Audio en Android con el uso de un Soundpool.
- **AndroidSound:** Clase que genera un sonido en Android (*llamada desde AndroidAudio*).
- **AndroidFile:** Clase que gestiona archivos .JSON: lee y borra archivos, y carga y guarda información encriptada con un hash.
- **AndroidShare:** Clase que permite crear una captura de pantalla y compartirla por medio de intents
- **AndroidWorker:** Clase que gestiona las notificaciones que se van a lanzar.
- **AndroidEngine.cpp:** Clase de C++ que permite encriptar datos.

APP implementa el juego en Android con su lógica y una clase MainActivity que lanza la actividad que hace que funcione la app. Dentro de este módulo es donde encontramos las distintas escenas del juego y las clases que implementan los elementos del mismo.

- **MenuScene:** Escena del menú principal, en ella puedes elegir el modo de juego (*partida rápida o modo aventura*), además de poder acceder a la tienda.
- **AdventureScene:** Escena del modo aventura, en ella puedes elegir el nivel al que quieres jugar.
- **ShopScene:** Escena de la tienda que permite desbloquear y seleccionar ítems usando diamantes.
- **DifficultyScene:** Escena para indicar la dificultad de las oleadas de la partida a jugar.
- **GameScene:** Escena que cubre el funcionamiento del juego.
- **FinalScene:** Escena que aparece si el jugador ha ganado la partida. En ella aparece el número de diamantes que has ganado en el nivel, además de la oportunidad de duplicarlas viendo un anuncio y compartir tu progreso en otras aplicaciones.
- **DiamondManager:** Gestiona todo el sistema de diamantes del juego (*sumar diamantes, restarlos...*).
- **GameLoader:** Gestiona el sistema de archivos usados en el juego, permitiendo guardar y cargar datos en un formato determinado (*carga de los ítems de la tienda, carga del estado del jugador: diamantes, ítems de la tienda seleccionados, carga de los niveles*).
- **LevelData:** Almacena la información necesaria para crear un nivel.
- **Tienda:** Tienda en la que puedes comprar tipos de torres y cosméticos para aquellas torres ya existentes. Cubre las siguientes clases:
 - **ShopManager:** Gestor de la tienda que accede al catálogo de ítems y al estado de progreso del jugador. También administra las compras y la selección de los ítems.
 - **ShopCatalog:** Catálogo de los diferentes ítems que contiene la tienda.
 - **PlayerShopState:** Gestor del estado de los ítems de la tienda.
 - **ShopItemData:** Información individual de cada ítem: tipo, id, precio, colores...
 - **ShopInfoPanel:** Panel que muestra la información del ítem seleccionado permitiendo su compra o selección.
- **Torres:** Tenemos varias clases que implementan las distintas torres del juego (*rayo, hielo, fuego...*) y sus interacciones con los enemigos.
- **Enemy:** Implementa el comportamiento de los enemigos y sus atributos.

Práctica 2: Cambios

Carga de niveles orientada a datos

Nombres de niveles ya no están hardcodedos.

AdventureScene.java

```
this.worldName = this.worlds.get(this.currentWorld).first;
```

Paso de mundos no limitado a 2 niveles.

AdventureScene.java

```
if (leftArrow.isTouched(e.x, e.y)) {
    if (currentWorld > 0) {
        currentWorld--;
        loadLevelStatesForCurrentWorld(this.currentWorld);
    }
}

if (rightArrow.isTouched(e.x, e.y)) {
    currentWorld++;
    if (currentWorld < this.gameLoader.get_totalWorlds()) {
        loadLevelStatesForCurrentWorld(this.currentWorld);
    }
    else
        currentWorld--;
}
```

Color de niveles no hardcodedo.

AdventureScene.java

```
this.graphics.setColor(this.gameLoader.get_worldcolor());
```

style.json

```
// En style.json de cada mundo hemos añadido "worldColor":
{
    "colorPassed":    "0xffb5532f",
    "colorUnlocked": "0xffffdd77",
    "colorLocked":   "0xff8f7a3e",
    "worldColor":    "0xffffdd77"
}
```

El motor ya no tiene que saber qué tipo de enemigos hay, los añade según lee el .JSON.

GameLoader.java

```
for (int i = 0; i < wavesJson.length(); i++) {
    JSONObject waveObj = wavesJson.getJSONObject(i);
    Iterator<String> keys = waveObj.keys();
    String enemyType = keys.next();
    waveTypes.add(enemyType);
    waveAmounts.add(waveObj.getInt(enemyType));
}
```

```
}
```

Cosméticos (*fondo*) en tienda

Añadidos los cosméticos en el .JSON y aparecen como opción comprable en la tienda, con su respectivo icono y precio. Hemos hecho la gestión correspondiente en los archivos *GameLoader*, *PlayerShopState*, *ShopItemData*, *ShopManager* y *ShopScene*.

style.json

```
{
  "id": "colorRojo",
  "type": "FONDO",
  "cost": 100,
  "description": "Nuevo color para el fondo de la aplicación y los botones.",
  "image": "sprites/rojo.png",
  "backgroundColor": "0xffff596a5",
  "buttonColor": "0xffeb0a2d",
  "buttonColor2": "0xffffcccd4"
},
{
  "id": "colorVerde",
  "type": "FONDO",
  "cost": 100,
  "description": "Nuevo color para el fondo de la aplicación y los botones.",
  "image": "sprites/verde.png",
  "backgroundColor": "0xffb7e4c7",
  "buttonColor": "0xff2d6a4f",
  "buttonColor2": "0xffd8f3dc"
},
{
  "id": "colorAzul",
  "type": "FONDO",
  "cost": 100,
  "description": "Nuevo color para el fondo de la aplicación y los botones.",
  "image": "sprites/azul.png",
  "backgroundColor": "0xffcde7ff",
  "buttonColor": "0xff1d4ed8",
  "buttonColor2": "0xffe0f2fe"
}
```

Corregido el scroll (*tienda*)

Añadidas variables para el scroll y ajustado para que sea fluido.

ShopScene.java

```
case TOUCH_DOWN:
    lastTouchY = e.y;
    isScrolling = true;
    break;
case TOUCH_MOVE:
    if (isScrolling) {
```

```

int deltaY = lastTouchY - e.y;
lastTouchY = e.y;

scrollOffset += deltaY;
scrollOffset = Math.max(minScrollOffset, Math.min(maxScrollOffset, scrollOffset));

for (Map.Entry<String, Button> b : towerButtons.entrySet()) {
    b.getValue().setY(og_y1 - scrollOffset);
}
for (Map.Entry<String, Button> b : skinButtons.entrySet()) {
    b.getValue().setY(og_y2 - scrollOffset);
}
for (Map.Entry<String, Button> b : colorButtons.entrySet()) {
    b.getValue().setY(og_y3 - scrollOffset);
}
y1 = og_y1 - scrollOffset;
y2 = og_y2 - scrollOffset;
y3 = og_y3 - scrollOffset;
}
break;

```

Archivos de mundos

Algunos archivos estaban repetidos o no conectaban bien el camino. Cambios hechos en los archivos *level_X_XX.json* de los assets.

Notificaciones

Para que solo haya una única notificación y no se sobreponga la misma, usamos una política de REPLACE cuando encolamos la notificación. También funciona abrir la aplicación desde la notificación.

AndroidEngine.java

```

private void programNotification(Long time, String title, String desc, int icon, String
mainName) {

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        CharSequence name = "Notificacion Programada";
        String description = "description";
        int importance = NotificationManager.IMPORTANCE_DEFAULT;
        NotificationChannel channel = new NotificationChannel(channelId, name, importance)
;

        channel.setDescription(description) ;
        NotificationManager notificationManager = (NotificationManager)
activity.getBaseContext().getSystemService(Context.NOTIFICATION_SERVICE); //NotificationMa
nagerCompat.from(getApplicationContext());
        notificationManager.createNotificationChannel(channel) ;
    }

    OneTimeWorkRequest notificationWork = new
OneTimeWorkRequest.Builder(AndroidWorker.class)
        .setInputData(new Data.Builder()
            .putString("title", title)

```

```

        .putString("description", desc)
        .putInt("notifications_icon", icon)
        .putString("notifications_channel_id", channelId)
        .putString("class", mainName)
        .build()
    .setInitialDelay(time, TimeUnit.SECONDS)
    .build();

    WorkManager.getInstance(activity.getBaseContext()).enqueueUniqueWork("notification_work",
    ExistingWorkPolicy.REPLACE, notificationWork);
}

```

Rewarded Ads

Cambios en el callback para que la cantidad recompensada sea correcta.

FinalScene.java

```

if(adButton.isTouched(e.x, e.y) && win)
{
    if(!adWatched && firstTime)
    {
        androidAds.showRewardedAd(() -> {
            this.diamondsPerLevel = this.diamondsPerLevel * 2;
            DiamondManager.addDiamonds(this.diamondsPerLevel);
            gameLoader.saveDiamonds(DiamondManager.getDiamonds());
            adWatched = true;
        });
    }
    else if(!adWatched && !firstTime)
    {
        androidAds.showRewardedAd(() -> {
            DiamondManager.addDiamonds(this.diamondsPerLevel);
            gameLoader.saveDiamonds(DiamondManager.getDiamonds());
            adWatched = true;
        });
    }
}
}

```

Ajustes de selección (*tienda*)

Corregida la selección y deselección de un ítem en la tienda, así como los sprites de candado que aparecían cuando ya estaban comprados.

ShopManager.java

```

private void deselect(ShopItemData item) {
    if (item.getType() == ShopItemData.ShopItemType.TOWER) {
        playerShopState.deselectTower(item.getId());
    }
    else if (item.getType() == ShopItemData.ShopItemType.SKIN) {
        playerShopState.deselectSkin(item.getId());
    }
}

```

```

        else if (item.getType() == ShopItemData.ShopItemType.FONDO) {
            playerShopState.selectColor(null);
        }
    }

    /**
     * Método que gestiona la selección/deselección de un ítem al comprarlo.
     * @param itemId
     */
    public boolean toggleSelectItem(String itemId) {
        if (!catalog.exists(itemId)) return false;
        if (!playerShopState.isPurchased(itemId)) return false;

        ShopItemData item = catalog.getItem(itemId);

        if (isSelected(itemId)) {
            deselect(item);
        } else {
            autoSelect(item);
        }

        return true;
    }
}

```

Ajustes en los anuncios

La carga de anuncios ahora se hace por listeners.

AndroidAds.java

```

public void loadRewardedAd(String adId) {
    rewardedAdID = adId;
    RewardedAd.load(activity, adId, new AdRequest.Builder().build(), new
    RewardedAdLoadCallback() {
        @Override
        public void onAdLoaded(RewardedAd ad) {
            rewardedAd = ad;
            rewardedAd.setFullScreenContentCallback(new FullScreenContentCallback() {
                @Override
                public void onAdDismissedFullScreenContent() {
                    rewardedAd = null;
                    if (rewardedAdID != null)
                        loadRewardedAd(rewardedAdID);
                }
                @Override
                public void onAdFailedToShowFullScreenContent(@NonNull AdError adError) {
                    rewardedAd = null;
                    if (rewardedAdID != null)
                        loadRewardedAd(rewardedAdID);
                }
            });
        }

        @Override
        public void onAdFailedToLoad(@NonNull LoadAdError loadAdError) {
            rewardedAd = null;
        }
    });
}

```